

DBMS MINI PROJECT · 2026

Hotel Reservation System

A fully functional relational database designed for hotel operations — covering room booking, customer management, and billing using **Microsoft SQL Server**, **T-SQL**, and **SSMS**.

BY – Rishabh Srivastava



Problem Statement & Project Objectives



The Challenge

Design a database system to manage hotel operations — room bookings, customer records, and billing — with full data integrity and automation.

Objectives

- ❌ Design a **normalized, scalable** schema
- ❌ Enforce **data integrity** with constraints
- ❌ Support **efficient querying** and reporting
- ❌ Automate logic with **triggers**
- ❌ Implement **transaction control** and security

Database Schema & Table Structure

Three normalized tables with clear primary/foreign key relationships, achieving **BCNF (3NF+)**.

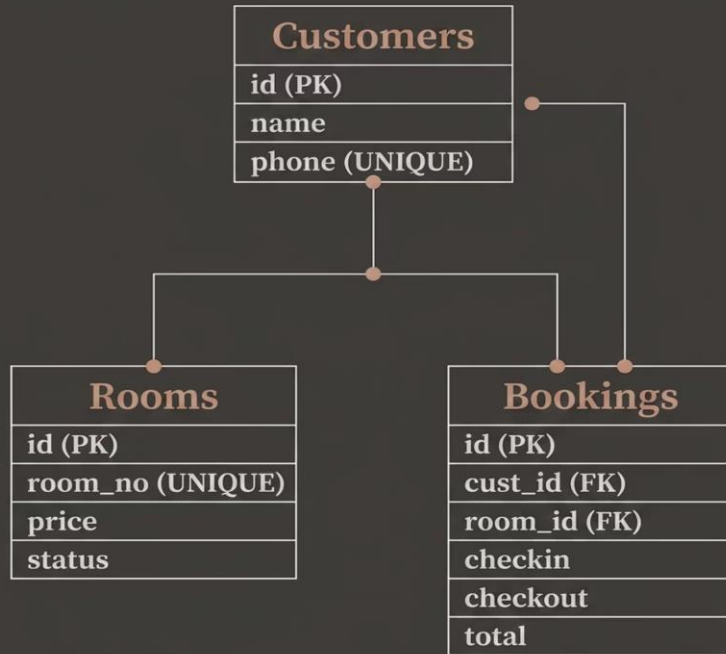


Table Relationships

Customers → Bookings — One customer can have many bookings (1:M)

Rooms → Bookings — One room can be booked many times (1:M)



Normalization achieved up to **BCNF**, eliminating insertion, update, and deletion anomalies.

Core & Advanced SQL Features

DDL & DML

CREATE, DROP, ALTER · INSERT, UPDATE, DELETE

Constraints & Keys

PRIMARY KEY, FOREIGN KEY · UNIQUE, NOT NULL, CHECK

Views & Functions

vw_AvailableRooms, vw_AllBookings · fn_Nights

Triggers & Joins

trg_CalcTotal, trg_UpdateRoom · INNER JOIN, LEFT JOIN

Subqueries & Aggregation

Subqueries, GROUP BY, HAVING

TCL & DCL

COMMIT, ROLLBACK · GRANT, REVOKE

7 SQL script files · 30+ operations tested end-to-end.

Key Query Examples

Query 1 — INNER JOIN

```
SELECT c.name, r.room_no
FROM Customers c
INNER JOIN Bookings b
    ON c.id = b.cust_id
INNER JOIN Rooms r
    ON b.room_id = r.id
```

Output: Raj → Room 101 · Priya → Room 201

Query 2 — VIEW

```
SELECT * FROM vw_AvailableRooms
```

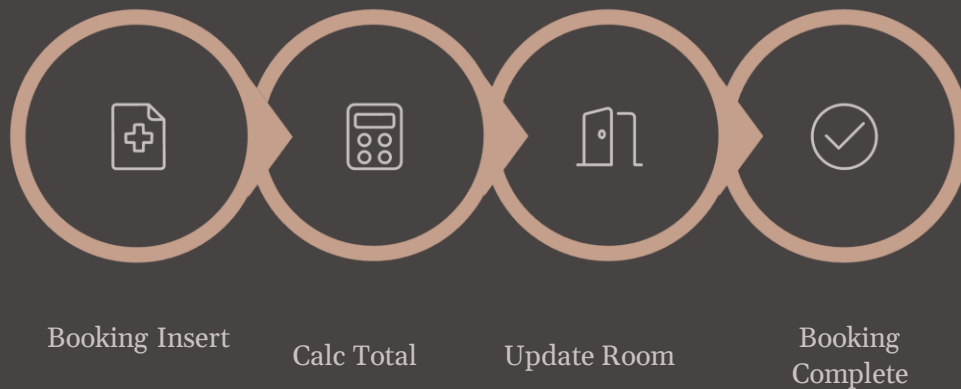
Output: Rooms 101, 102, 201, 202, 301 with prices

Query 3 — SUBQUERY

```
SELECT name FROM Customers
WHERE id IN (
    SELECT cust_id FROM Bookings)
```

Output: Raj, Priya, Amit

Functions, Triggers & Automation



The diagram above illustrates the automated trigger chain that fires on every new booking — calculating the total cost and updating room availability without manual intervention.

fn_Nights

Input: Start date, end date

Output: Number of nights

Example: `fn_Nights('2026-06-01', '2026-06-03')` → **2 nights**

trg_CalcTotal (*BEFORE INSERT*)

Auto-calculates **total = Nights × Room Price** on every new booking. The total column is filled automatically — no manual entry needed.

trg_UpdateRoom (*AFTER INSERT*)

Updates room status from '**Available**' → '**Booked**' immediately after a booking is confirmed, preventing double-booking conflicts.

Transaction Control & Data Security

TCL — COMMIT

```
BEGIN TRANSACTION;  
INSERT INTO Bookings  
  VALUES (3, 104, ...);  
COMMIT;
```

Changes are **permanently saved** to the database.

TCL — ROLLBACK

```
BEGIN TRANSACTION;  
DELETE FROM Bookings  
  WHERE id = 3;  
ROLLBACK;
```

Deletion is **cancelled**; data restored to prior state.

DCL — User Access Control

```
CREATE USER hotel_user  
  WITHOUT LOGIN;  
  
GRANT SELECT ON vw_AvailableRooms  
  TO hotel_user;  
  
REVOKE INSERT ON Customers  
  FROM hotel_user;
```

The `hotel_user` account has **read-only access** to availability reports and **cannot insert** into customer tables — enforcing least-privilege security.

Summary & Future Scope

✓ Delivered

- Schema normalized to **BCNF**
- **7 scripts** · 30+ operations
- DDL, DML, TCL, DCL complete
- 2 triggers + 1 function automated
- Indexes for query performance

🔮 Future Scope

- Online **payment gateway** integration
- Real-time **availability dashboard**
- **Mobile app** connectivity
- Customer **loyalty program**

📁 📁 Project Repository: [GitHub/P4_Hotel_Reservation_System](#) · Thank you!

